



Web

Projet Sampler Audio/Angular

Auteurs :

Bensallah Younes, Ouharani Sofiane

Université Côte d'Azur

Master 1 Informatique

Année universitaire 2025–2026

Date de rendu : 30 Janvier 2026

Table des matières

1	Introduction	2
2	Architecture du projet	2
2.1	Organisation des fichiers	2
3	Fonctionnalités principales	2
3.1	Front-end Sampler	2
3.2	Back-end API	3
3.3	Admin Angular	3
4	Utilisation de l'intelligence artificielle	4
5	Guide d'utilisation	4
5.1	Installation Locale	4
5.2	Utilisation du sampler	4
6	Tests et validation	5
7	Limites et perspectives	5

1 Introduction

Ce rapport présente le développement d'une application web de sampling audio réalisée dans le cadre du module de Web. L'objectif est de permettre la création, l'édition et la gestion de kits de sons, avec une interface propre et des fonctionnalités avancées vraiment utiles pour les personnes concernées (musiciens, beatmakers par exemple).

2 Architecture du projet

Le projet est composé de trois parties principales, conçues pour être indépendantes les unes des autres :

- **Front-end Sampler** : C'est le cœur de l'application. Il contient toute la logique audio (Web Audio API) et l'interface utilisateur (HTML/CSS/JS) pour jouer de la musique.
- **Back-end API** : Un serveur Node.js relié à une base de données MongoDB. Il sert de "mémoire" à l'application en stockant la liste des kits (presets) et les fichiers sons.
- **Admin Angular** : Une application de gestion séparée qui permet d'administrer les données (renommer ou supprimer des kits) sans passer par le sampler.

2.1 Organisation des fichiers

```
ProjetWeb/  
  api/           (Backend Node.js + Fichiers sons)  
  front/         (Frontend Sampler - Site Web)  
  sampler-admin/ (App Angular - Interface Admin)
```

3 Fonctionnalités principales

3.1 Front-end Sampler

C'est ici que se concentre la majorité de notre travail technique. Nous avons utilisé une architecture modulaire en JavaScript moderne (ES6).

- **Architecture Modulaire** : Nous avons séparé le code en plusieurs fichiers (Engine, GUI, Sequencer, MIDI) pour que chaque partie ait un rôle précis.
- **Pads interactifs** : 24 pads configurables qui réagissent au clic et au clavier.
- **Moteur Audio Avancé** : Utilisation de l'AudioContext pour créer une chaîne d'effets complexe.
- **Effets audio globaux** :
 - Distorsion (WaveShaper) pour salir le son.
 - Filtre (BiquadFilter) pour étouffer les aigus.
 - Reverb (Convolver) pour donner de l'espace.
 - Vitesse de lecture variable.
- **Effets par pad** : Chaque son peut être réglé individuellement (volume, panoramique gauche/droite, sens de lecture inversé, hauteur de note).
- **Visualisation waveform** : On dessine la forme d'onde du son sur un canvas, ce qui permet de sélectionner précisément le début et la fin du sample (trimming).

- **Séquenceur** : Une grille de 16 pas qui permet de programmer des boucles rythmiques.
- **Enregistrement micro intelligent** : On peut enregistrer sa voix. Si le système détecte des silences, il découpe automatiquement l'enregistrement en plusieurs morceaux et les place sur différents pads.
- **Recherche Freesound** : On peut chercher des sons directement sur la banque de données Freesound.org via leur API et les importer dans le kit.
- **Sauvegarde de presets** : Le bouton "Sauvegarder" envoie le fichier audio et les infos du kit au serveur pour ne rien perdre.

Support MIDI et Mode Headless

Pour la gestion du MIDI, nous sommes partis de l'exemple du cours (JSBin) mais nous l'avons totalement refait en Programmation Orientée Objet. Notre classe `MidiController` permet de brancher un clavier physique USB.

- **Architecture par événements** : Au lieu de tout coder en dur, on peut "abonner" des fonctions à des notes précises.
- **Mode Headless** : Comme le contrôleur MIDI parle directement au moteur audio sans passer par l'interface graphique, cela prouve que notre code respecte la consigne de séparation GUI/Moteur. Si on supprime l'affichage, le son marche quand même.

3.2 Back-end API

Pour le serveur, nous avons choisi une architecture MVC (Modèle-Vue-Contrôleur) simple et efficace.

- **API REST** : Des routes claires (GET, POST, DELETE) pour gérer les presets.
- **Stockage Hybride** :
 - Les **fichiers sons** (.wav, .mp3) sont stockés sur le disque du serveur pour être servis rapidement.
 - Les **informations** (nom du kit, réglages) sont stockées dans **MongoDB Atlas** (Cloud) pour la sécurité et la flexibilité.
- **Upload** : Utilisation de la librairie Multer pour recevoir les fichiers envoyés par le front-end.

3.3 Admin Angular

L'application Angular sert de tableau de bord pour gérer la base de données.

- **Liste des presets** : On voit tous les kits disponibles, triés par date.
- **Création rapide** : On peut créer un kit en donnant des URLs de sons.
- **Renommage inline** : On peut cliquer sur le nom d'un kit pour le modifier directement sans changer de page.
- **Suppression** : Permet de nettoyer la base de données.
- **Simplification** : Nous avons retiré la gestion complexe des catégories pour simplifier : soit un kit est "Usine" (non modifiable), soit c'est un kit "Custom" (utilisateur).

Déploiement sur Render

Le back-end Node.js du projet a été déployé sur la plateforme cloud Render. Ce déploiement permet d'accéder à l'API et aux fonctionnalités du sampler depuis n'importe où, sans avoir à lancer le serveur localement.

Accès : L'API est accessible à l'adresse suivante :

`https://projet-sampler-audio-angular.onrender.com/api/`

La procédure d'installation sur Render a consisté à :

- Créer un compte sur Render et connecter le dépôt GitHub du projet.
- Configurer les variables d'environnement (URI MongoDB, PORT, etc.) pour sécuriser les accès.
- Définir le script de démarrage (`npm start`).
- Lancer le déploiement automatique à chaque push sur la branche principale.

4 Utilisation de l'intelligence artificielle

- **GitHub Copilot** : autocomplétion et suggestions de code.
- **Claude / Gemini** : aide à l'architecture et au debugging.

Nous avons utilisé l'IA comme un assistant pour :

- Comprendre comment refactoriser le code MIDI procédural en classe objet.
- Nous aider à déboguer plusieurs problèmes.
- Faire de l'autocomplétion à certains endroits, durant la réalisation du projet.

5 Guide d'utilisation

5.1 Installation Locale

1. Installer Node.js (18+), npm ou yarn.
2. Cloner le projet et installer les dépendances dans chaque dossier (`api/` et `sampler-admin/`) avec la commande `npm install`.
3. Configurer le fichier `.env` dans le dossier `api/` avec vos identifiants MongoDB Atlas.
4. Lancer le serveur API : `npm start` dans le dossier `api/`.
5. Lancer le Sampler : Ouvrir le fichier `front/index.html` avec "Live Server" ou un serveur HTTP local.
6. Lancer l'admin Angular : `ng serve` dans le dossier `sampler-admin/`.

5.2 Utilisation du sampler

- **Jouer** : Cliquez sur les pads ou utilisez les touches du clavier (A à N).
- **Charger un son** : Utilisez le menu déroulant "Kits" ou cherchez un son sur Freesound via la barre de recherche.
- **Modifier** : Cliquez sur un pad pour ouvrir l'éditeur (en bas). Vous pouvez changer le volume, le pitch, ou couper le début/fin du son.
- **Enregistrer** : Cliquez sur "REC" pour enregistrer votre voix. Si vous faites des pauses, le logiciel coupera automatiquement les silences.
- **Séquenceur** : Activez les cases de la grille pour créer un rythme, puis appuyez sur Play.

6 Tests et validation

Nous avons mis en place plusieurs niveaux de tests pour garantir la qualité :

- **Test headless (Fichier `test_headless.html`)** : C'est une page spéciale qui lance le moteur audio sans charger l'interface graphique. Elle exécute une série de tests automatiques (création du contexte audio, chargement des sons...) et affiche des coches vertes si tout va bien. Cela prouve que notre code est bien découplé.
- **Tests manuels** : Nous avons vérifié chaque fonctionnalité (pads, effets, sauvegarde) à la main.
- **Tests API** : Nous avons vérifié que le serveur renvoie bien les données JSON correctes.

7 Limites et perspectives

Bien que fonctionnel, le projet pourrait être amélioré :

- L'upload de fichiers audio dans l'admin Angular est limité aux URLs (on ne peut pas envoyer un fichier MP3 depuis l'admin pour l'instant).
- L'auto-slice du micro est basique (basé sur le volume), il pourrait être plus intelligent.
- On pourrait transformer toute l'interface en Web Component pour pouvoir l'intégrer facilement dans d'autres sites.